Staged Reward Shaping for Delayed Strategic Objectives in Generals.io

Zemu Zhu

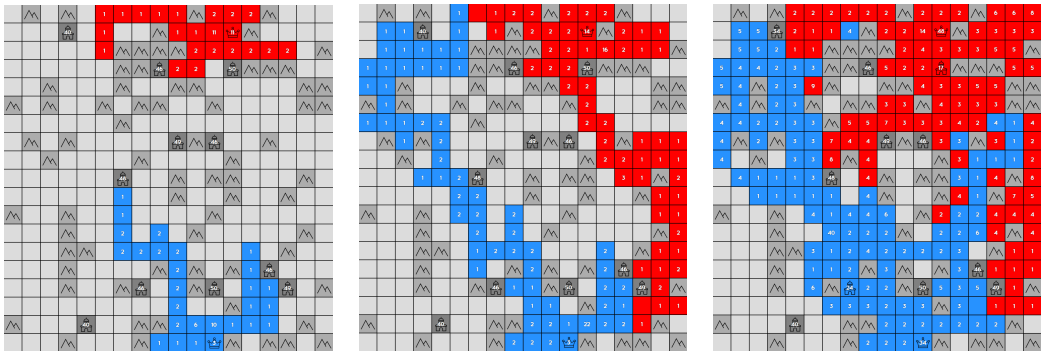
Abstract

Generals.io is a partially observable real-time strategy game in which agents must expand territory, manage armies, capture cities, and eventually defeat the opponent's general. We study temporal credit assignment for delayed strategic objectives in this setting, using neutral city capture as a measurable case study. Capturing a city immediately consumes army, while the benefit arrives slowly through future production. A terminal win-loss objective can train strong attack-oriented policies, but in our experiments it often underlearns this delayed-return behavior. A naive city-capture reward overcorrects by increasing captures while harming combat strength. We therefore use a two-level staged reward design: reward weights vary across game phases, and training proceeds through a curriculum that first preserves the delayed behavior, then provides intermediate signal for useful investment, and finally halves the shaping pressure while penalizing failed city attacks. In local fixed-size simulator evaluations, the final model improves over retained internal PPO baselines. It captures cities less frequently than the high-frequency city-reward baseline, but its city attacks have higher success and stronger conditional win rates, suggesting more selective city use. In a small online private-room check against Human.exe, a top-level community bot, the final model scores 16–24, compared with 3–15 for a terminal-only PPO baseline.

1 Introduction

Generals.io is a compact but challenging real-time strategy game. Each player starts with one general, observes only part of the map, and repeatedly moves armies between adjacent cells. The player loses when the general is captured, while terrain, cities, fog of war, and long-horizon army growth make the game difficult for simple search or myopic policies.

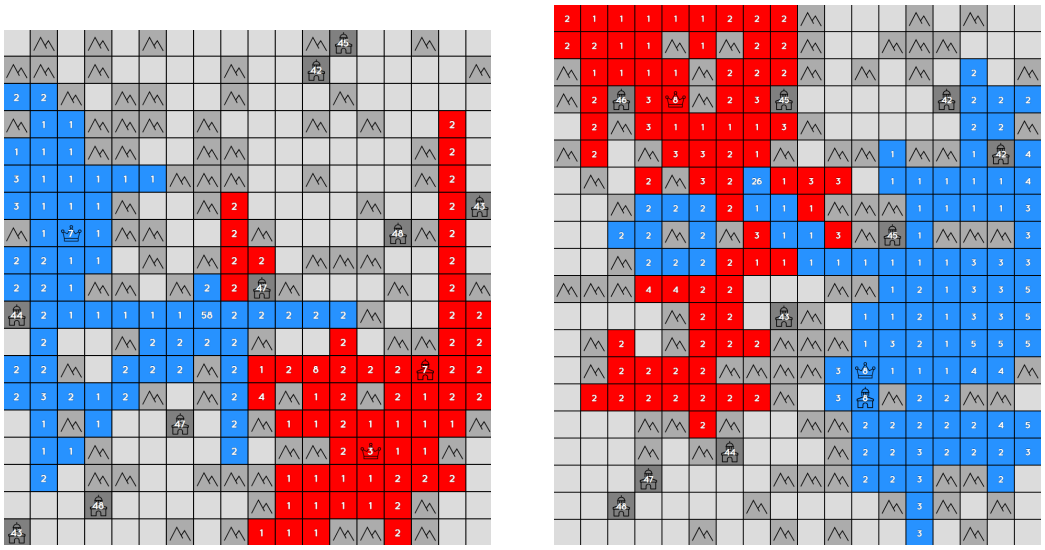
Figure 1 shows how a typical local match evolves. In the opening, both players mainly expand from their generals and collect neutral land. In the middle game, the front line becomes visible and players must decide how to allocate armies between expansion, defense, and pressure on the opponent. In the late game, the map is largely claimed, so combat positioning, city ownership, and general safety dominate the outcome. This progression motivates our focus on both early development and later strategic investment.



(a) Opening (b) Middle game (c) Late game

Figure 1: A Generals.io match evolves from early expansion to contested middle-game fronts and late-game army concentration. Gray cells are unclaimed or terrain, colored cells are player territories, crown icons mark generals, and tower icons mark cities.

Within this progression, neutral city capture is a clean and measurable example of a delayed-return objective. A city is valuable because it becomes a persistent source of army production after capture. However, capturing it requires spending a large army immediately, and that army is temporarily unavailable for expansion, defense, or attacks on the opponent. City capture therefore has an immediate army cost and a delayed production return. Capturing too early can lose the game by weakening the front line; ignoring cities can lose the game later by falling behind in production. Figure 2 contrasts these two outcomes. Our goal is therefore not to maximize the number of city captures, but to train an agent that balances immediate tactical risk against long-term economic return.



(a) Premature capture (b) Well-timed capture

Figure 2: Neutral city capture illustrates the risk-return tradeoff behind delayed strategic credit assignment. Capturing a city at the wrong time can consume army and create immediate tactical pressure, while a well-timed capture can become a persistent production advantage and contribute to winning many turns later.

Motivation. The city examples above illustrate a broader temporal credit-assignment problem in partially observable, long-horizon strategy games. Some actions, such as attacking an exposed enemy cell or pressuring the opponent's general, can affect the outcome quickly and therefore provide a relatively clear learning signal under self-play. Other actions are better understood as strategic

investments: they may reduce short-term army, territory, or pressure, while improving the player's position many turns later. When training only from terminal win-loss rewards, a policy may therefore learn behaviors whose consequences are easier to attribute, such as direct combat or general pressure, while underlearning slower development behaviors whose benefits are delayed. This is a common difficulty in strategy games and long-horizon reinforcement learning more broadly: sparse terminal rewards can favor visibly aggressive routes to victory while failing to credit earlier development choices that made later wins possible.

Recent work introduced a Generals.io reinforcement learning benchmark and a strong reference agent trained with supervised pre-training, self-play, memory features, and reward shaping [10]. Their paper also identifies Human.exe as a previous best-known community bot. Our project follows a smaller-scale but similar direction: we build a local simulator, train neural agents from replay data, improve them with PPO [7], and evaluate them through local head-to-head matches plus smaller online checks against Human.exe [2].

This motivates the main empirical question in the paper. Terminal-only PPO learns combat-oriented policies but underuses this delayed-return objective. Conversely, a naive city reward overcorrects: it increases city captures, but does not reliably teach when captures are useful, and the resulting policies can over-invest in cities and lose combat strength.

We therefore use staged city-aware shaping as the central design. The word staged has two meanings in this work. Within a game, reward weights depend on the turn range, so the opening can emphasize expansion, the middle game can balance development and combat, and the late game can give more credit to durable strategic investments. Across training, the policy moves through a curriculum: behavior cloning gives basic movement and expansion, PPO gives combat skill, city-focused shaping provides an intermediate learning signal for delayed city value, lifecycle rewards discourage fragile captures, and a final low-shaping continuation adds a penalty for failed neutral-city attacks. The curriculum is not intended as a theoretical guarantee of optimality. The final policy is best understood as a lower-frequency city-capture policy whose captures are more correlated with eventual wins, rather than as causal proof that every capture decision is better.

We evaluate this balance through three main axes: head-to-head win rate, turn-50 opening land as an early-development diagnostic, and neutral-city behavior quality measured by attack frequency, capture success, and capture-conditioned outcomes. We additionally use Flobot, Human.exe, and small human private-room records as external calibration rather than as controlled leaderboard evidence.

Our contributions are:

- We implement a complete local Generals.io training and evaluation pipeline with replay-based behavior cloning, self-play PPO, local visualization, and head-to-head evaluation.
- We identify a delayed-objective blind spot of terminal-only self-play: strong attack-oriented agents can underuse strategic investments whose payoff arrives many turns later.
- We use neutral city capture as a concrete, measurable instance of this problem and show that simple city rewards increase captures without reliably solving the timing problem.
- We propose and evaluate a staged reward-shaping curriculum for balancing immediate combat risk, development, and delayed strategic value.
- We calibrate learned agents against external opponents such as the public Flobot heuristic and Human.exe, a top-level community bot used as a stronger online benchmark.

2 Related Work

Generals.io reinforcement learning. Straka and Schmid [10] introduced a Gymnasium- and PettingZoo-compatible Generals.io environment and trained a strong agent with supervised pre-training, self-play, reward shaping, and memory features. Their work motivates our setup and provides the main reference point for the task. Earlier Generals.io work includes HASP, a hierarchical self-play approach that reported a 77% win rate against Flobot [11], and Generally Genius, a bot-development and data-collection framework that analyzed Flobot as a top-performing rule-based agent [1]. Our project is smaller in compute and scope, but focuses on an interpretable reward-shaping failure mode and its empirical mitigation.

Imitation learning and self-play. We use behavior cloning from replay data as an initialization step. This is a standard imitation learning approach, although pure imitation can suffer from distribution shift when the learned policy visits states that are rare in the demonstrations [6]. Self-play has been highly successful in board games and adversarial environments [8]; in our setting, self-play PPO is used to improve beyond the supervised baseline.

Reward shaping. Reward shaping can accelerate learning, but it can also change the effective objective if not designed carefully. Potential-based shaping has known policy-invariance properties [4]. Our shaping rewards are heuristic rather than theoretically invariant, so we treat them as a curriculum mechanism and explicitly evaluate the tradeoff between shaped behavior and final win rate.

Spatial neural policies. The policy operates on grid observations, so we use a convolutional encoder with a U-Net-like spatial policy head [5]. This preserves spatial locality and naturally produces move logits over board locations and directions.

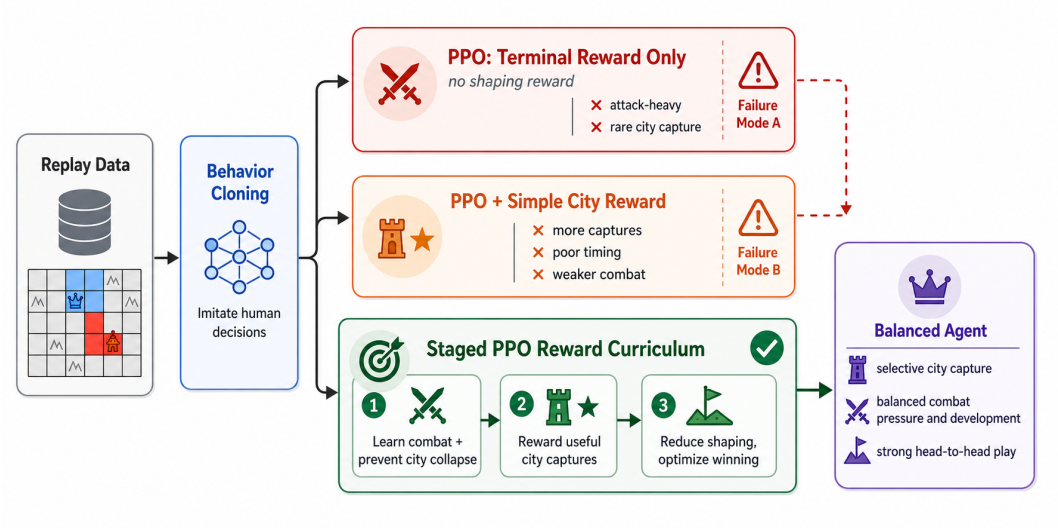


Figure 3: Overview of the training curriculum and observed failure modes. After behavior cloning, all branches use PPO but differ in their reward design. Terminal-only PPO learns attack-heavy policies that underuse the delayed city objective, while a simple city-capture reward increases captures but degrades timing and combat strength. The proposed staged curriculum treats city capture as a measurable delayed strategic investment: it preserves combat, improves useful captures, and then reduces shaping pressure while penalizing failed city attacks to obtain a balanced agent.

3 Method

3.1 Environment and Observation

We use a local simulator for two-player Generals.io-style matches. The map is a fixed 18×18 generated board in the main evaluations. Each player observes its own territory, visible neighboring cells, known terrain, army counts for visible cells, and public aggregate information such as turn number. The policy outputs either pass or a move from a source cell to an adjacent target cell. Invalid moves are masked during inference.

Each observation frame is encoded into 22 spatial feature planes: ownership and visibility planes, terrain/city/general indicators, log-scaled visible army counts, public own/enemy total army and land, and turn features. The neural policy receives a history of $H = 8$ frames by channel concatenation:

$$x_t = [\phi(o_{t-7}), \ldots, \phi(o_t)] \in \mathbb{R}^{176 \times 18 \times 18}.$$

When fewer than eight observations are available at the start of an episode, the earliest frame is repeated. This history stack is used by all behavior-cloning and PPO agents reported in this paper.

Most controlled evaluations in this paper use local simulator matches rather than the public online server. This makes the main comparisons reproducible and fast. We additionally run small online private-room checks against Human.exe and human players as external calibration; those online records should be interpreted as coarse transfer evidence rather than controlled leaderboard evaluations.

3.2 Network and Policy Parameterization

The policy network is a small U-Net-style convolutional model. It uses two stride-2 downsampling blocks, a bottleneck residual block, and two nearest-neighbor upsampling blocks with skip connections. With base width 64, the channel widths are $64 \to 128 \to 192 \to 128 \to 64$. All convolutional blocks use GroupNorm and SiLU activations. The policy and value heads share this encoder.

Actions are represented hierarchically. Let $\mathcal{M}(s_t)$ be the legal spatial moves under the current observation and let $a_\emptyset$ denote pass. The network outputs a scalar action logit $g_\theta(s_t)$ and spatial move logits $m_\theta(s_t, u, d)$ for each source tile u and direction d. The policy is

$$\pi_\theta(a_\emptyset \mid s_t) = 1 - \sigma(g_\theta(s_t)),$$

$$\pi_\theta((u,d) \mid s_t) = \sigma(g_\theta(s_t)) \frac{\exp m_\theta(s_t, u, d)}{\sum_{(u',d') \in \mathcal{M}(s_t)} \exp m_\theta(s_t, u', d')}, \quad (u,d) \in \mathcal{M}(s_t).$$

If no legal move exists, the action is forced to pass. The value head is a global-average-pooled MLP over the final U-Net feature map and predicts a player-relative scalar value.

3.3 Behavior Cloning Baseline

The first stage trains a neural policy from public replay data [9]. For each player turn, replay actions are converted into supervised action labels. We use cross-entropy for the hierarchical action decision and a small distance-aware loss to reduce the penalty for spatially nearby move errors. Late-game pass labels are down-weighted in behavior cloning to avoid a degenerate policy that stops moving after the opening.

The behavior cloning policy provides a useful initialization for self-play because it already expands, moves armies, and avoids the severe late-game pass collapse seen in earlier models.

3.4 Terminal-Only PPO

Starting from the behavior cloning policy, we train PPO agents with only terminal rewards:

$$r_T = \begin{cases} 1 & \text{win,} \\ -1 & \text{loss,} \\ 0 & \text{draw or unfinished.} \end{cases}$$

For a sampled action a_t, PPO uses the usual clipped objective with GAE advantages:

$$L_\pi = -\mathbb{E}_t \left[\min \left(\rho_t A_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) A_t \right) \right], \quad \rho_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}.$$

The optimized loss is

$$L = L_\pi + c_v L_V - c_H \mathcal{H}(\pi_\theta),$$

where L_V is the clipped value loss and $\mathcal{H}$ is the hierarchical policy entropy. Unless stated otherwise, PPO comparisons use the same training setup with 16 parallel environments, rollout length 800, four PPO epochs per update, learning rate 10^{-5}, entropy coefficient 0.001, and the 800-turn episode limit used in evaluation. Appendix A gives a compact hyperparameter summary.

This stage produces strong combat-oriented terminal-PPO policies. However, these models tend to underuse neutral cities. This is plausible because the policy receives only delayed credit for a successful city investment: the army loss is immediate, while the production advantage appears many turns later.

3.5 Staged Shaping for Delayed City Value

We use neutral city capture to operationalize the broader delayed-objective problem. The goal is not to reward every city attack, but to make the risk-return structure of a strategic investment learnable: the agent must spend army now for production value that may only matter many turns later. The most important component rewards successful neutral city capture, not merely attacking a city. Later variants also reward surviving after a capture and holding the city over a lifecycle window, because a city is only useful if the agent remains alive and can convert it into future production. We use these rewards only as a training signal; evaluation is still based on win rate and behavior metrics.

The word "staged" has two roles in our reward design. First, rewards are staged within a single game: the weights $w(t)$ depend on the turn range, so the opening can emphasize land and expansion, the middle game can balance development with combat pressure, and the late game can give more credit to durable strategic investments such as cities. Second, rewards are staged across training: later policies are initialized from earlier ones and the city-related shaping terms are changed to move from preserving city behavior, to increasing useful city captures, and finally back toward a lower-shaping win-oriented objective.

Let

$$D_i(z) = \log(1 + z_i) - \log(1 + z_{1-i})$$

be a relative log-score for player i and scalar public quantity z such as army, land, or owned-city count. The shaped reward for player i is

$$r_t^i = r_{\text{term}}^i + r_{\text{land}}^i + r_{\text{city}}^i + r_{\text{army}}^i + r_{\text{attack}}^i + r_{\text{cap}}^i + r_{\text{life}}^i + r_{\text{capwin}}^i + r_{\text{fail}}^i.$$

There is no turn penalty. The land term is a milestone reward paid every 50 turns,

$$r_{\text{land}}^i = \mathbb{1}[t \equiv 0 \pmod{50}]\, w_{\text{land}}(t) D_i(\text{land}_t),$$

while city ownership and army are potential-difference rewards,

$$r_z^i = w_z(t)\left(D_i(z_{t+1}) - D_i(z_t)\right), \quad z \in \{\text{city}, \text{army}\}.$$

The attack reward is paid only for damage to enemy-owned cells, $w_{\text{attack}}(t)\log(1 + \text{damage})$, with the opposite sign for the damaged player. It is not a reward for failed attacks on neutral cities.

The neutral-city capture reward is paid only when a valid move captures a neutral city immediately:

$$r_{\text{cap}}^i = w_{\text{cap}}(t)\left(1 + \frac{\log(1 + a_{\text{city}})}{\log(51)}\right)\mu(d),$$

where a_{city} is the target city's army before capture and $\mu(d)$ is an optional safety multiplier based on the game-graph distance d from the captured city to the nearest enemy tile. When enabled, $\mu(d) = 0.25$ for $d \leq 3$, $\mu(d) = 1$ for $d \geq 10$, and is linearly interpolated between these distances. The opponent receives the negative of this immediate capture reward.

For lifecycle shaping, a successful city capture at turn τ creates an event with window W. For each turn $\tau < t \leq \tau + W$, the capturer receives a survival drip reward if alive and a hold drip reward if the captured city is still owned. At $t = \tau + W$, the capturer receives survival and hold milestones if the respective conditions remain true. If the capturer dies before the window ends, a death penalty is applied. A separate terminal bonus, when enabled, gives the winner $\beta\log(1 + N_{\text{cap}})$, where N_{cap} is the number of successful neutral-city captures by that winner. The final stage additionally uses a failed-city-attack penalty: if a neutral-city attack does not lead to owning that city within 10 turns, the attacking player receives $r_{\text{fail}} = -0.02$.

The curriculum is:

1. Start from the behavior cloning policy.
2. Train PPO policies that learn combat while preventing city behavior from collapsing.
3. Add heuristic city-capture rewards that favor useful captures instead of failed attacks.
4. Add lifecycle rewards so that the agent is rewarded for capturing cities it can keep and survive with.
5. Continue from the lifecycle policy with all stable city-shaping weights reduced to 50%, and add the failed-city-attack penalty above to suppress low-quality city crashes.

The final stage gives the Step 1+2+3 policy. The design goal is not to maximize the number of city captures, but to learn when the immediate army cost is justified by future production and when a city attack should be avoided.

4 Experiments

4.1 Evaluation Protocol

Unless otherwise stated, local head-to-head strength evaluation uses fixed 18×18 generated maps, an 800-turn limit, no pass bias, and swapped-side games. A seed count of 100 corresponds to 200 games because each map is played with both side assignments. We report decided win rate as wins divided by non-draw, non-unfinished games.

For city behavior, we run longer 1200-turn self-play diagnostics and count, for each player-game, neutral city attacks and successful neutral city captures. We also report conditional outcome metrics:

$$P(\text{win} \mid \text{captured city}), \quad P(\text{loss} \mid \text{captured city}), \quad P(\text{draw/unfinished} \mid \text{captured city}).$$

These metrics help distinguish frequent but low-quality city capture from selective, useful city capture. This is important because a high capture count alone can indicate reward over-optimization rather than better play.

Finally, online Human.exe and human-player games are private-room calibration runs on the public server. They are useful transfer checks, but they are not a controlled ladder evaluation and have much smaller sample sizes than the local tables.

All experiment tables use the method names in Table 1.

Table 1: Method names used throughout the experiment tables.

Name	Role	Description
BC	Imitation baseline	Behavior cloning from public replays
Terminal PPO	Combat baseline	PPO from BC with terminal win/loss reward only
High city	Over-optimization control	Direct high city-capture reward
Step 1	Conservative city stage	Small city reward to preserve city behavior while learning combat
Step 1+2	Lifecycle city stage	Rewards useful captures through survival and holding windows
Step 1+2+3	Final model	Reduced city shaping plus failed-city-attack penalty

4.2 Method Ablation

Table 2 summarizes the main curriculum ablation as a round-robin-style matrix. Cells report the decided win rate of the row method against the column method; unfinished games are excluded from the denominator. The base ablation pairs use 1000 local games on fixed 18×18 generated maps with swapped sides and an 800-turn cap. The Step 1+2+3 row and column pool disjoint same-protocol follow-up evaluations against each retained earlier method; the corresponding sample sizes are shown in Table 3.

Table 2: Round-robin ablation. Each cell is the decided win rate of the row method against the column method. Mean WR is the unweighted mean of the five displayed non-diagonal cells in that row. Bold marks the best value in each column.

Method	BC	Terminal PPO	High city	Step 1	Step 1+2	Step 1+2+3	Mean WR
BC	–	0.286	0.356	0.206	0.283	0.257	0.278
Terminal PPO	0.714	–	0.587	0.461	0.443	0.406	0.522
High city	0.644	0.413	–	0.339	0.332	0.333	0.412
Step 1	**0.794**	0.539	0.661	–	0.466	0.441	0.580
Step 1+2	0.717	0.557	**0.668**	0.534	–	**0.478**	0.591
Step 1+2+3	0.743	**0.594**	0.667	**0.559**	**0.522**	–	**0.617**

The High city row uses a direct high city-capture shaping run. This direct reward increases city-oriented behavior but performs worse than Terminal PPO on average, illustrating reward over-

optimization. The staged rows recover strength gradually: Step 1 preserves combat while adding conservative city reward, Step 1+2 adds lifecycle-style city incentives, and Step 1+2+3 adds the final low-shaping failed-attack-penalty stage. The final row has the best mean win rate, but this matrix should be read as an aggregate comparison rather than a strict total ordering of policies. We therefore interpret the result as evidence for better overall robustness and behavior balance, not as proof that every added stage strictly dominates every predecessor by a large margin.

Because the main matrix combines many pairwise comparisons, Table 3 gives Wilson 95% confidence intervals for selected head-to-head estimates. These intervals quantify finite-match sampling noise for the displayed local evaluations only; they do not account for simulator mismatch, model selection, or other experimental-design uncertainty.

Table 3: Selected local head-to-head comparisons with Wilson 95% confidence intervals. Records are wins–losses–unfinished for the row method, and decided WR excludes unfinished games.

Method	Opponent	Record	Decided WR	Wilson 95% CI
Terminal PPO	BC	712–285–3	0.714	[0.685, 0.741]
High city	Terminal PPO	410–580–10	0.414	[0.384, 0.445]
Step 1+2+3	BC	368–127–5	0.743	[0.703, 0.780]
Step 1+2+3	Terminal PPO	404–276–20	0.594	[0.557, 0.630]
Step 1+2+3	High city	327–163–10	0.667	[0.624, 0.708]
Step 1+2+3	Step 1	381–301–18	0.559	[0.521, 0.595]
Step 1+2+3	Step 1+2	340–311–49	0.522	[0.484, 0.560]

The confidence intervals make this caution concrete. Step 1+2+3 is clearly stronger than BC, Terminal PPO, and High city in these local comparisons, but its direct margins over Step 1 and especially Step 1+2 are modest. The Step 1+2 comparison is best viewed as roughly tied in head-to-head strength, with the final stage mainly changing the city-use profile and improving the aggregate mean.

4.3 Opening Development

We use turn-50 land as a simple opening-development diagnostic. Table 4 reports archived self-play means for the methods in Table 1. Since each method is evaluated against itself, this table is not a controlled head-to-head opening benchmark; it only checks whether each method causes obvious early-expansion collapse.

Table 4: Representative turn-50 opening land for the methods in Table 1. Values are simulator self-play means and should be interpreted as diagnostics rather than controlled opponent comparisons. Bold marks the highest mean land.

Method	Mean land at turn 50
BC	22.96
Terminal PPO	22.53
High city	23.50
Step 1	23.20
Step 1+2	23.51
Step 1+2+3	**23.74**

4.4 City-Capture Behavior

The city argument requires two checks at the same time. Terminal PPO should actually lose the delayed city behavior, and staged shaping should recover useful city behavior without simply maximizing city captures. Table 5 therefore combines behavior frequency and capture-conditioned outcomes. All rows use long-horizon self-play diagnostics with a 1200-turn cap. Attacks and captures are averages per player-sample, N_C counts player-samples with at least one successful neutral-city capture, and $P(U \mid C)$ denotes draw or unfinished games conditioned on a successful city capture.

Table 5: Combined neutral-city behavior and outcome diagnostics. Avg. attacks and avg. captures are per-player-sample means. Success is captures divided by city attacks. N_C is the number of player-samples with at least one successful neutral-city capture. Rows with very small N_C have noisy conditional probabilities. Bold marks the best reliable quality metrics.

Method	Avg. attacks	Avg. captures	Success	N_C	$P(W \mid C)$	$P(L \mid C)$	$P(U \mid C)$
BC	0.8200	0.2900	0.354	40	0.450	0.550	0.000
Terminal PPO	0.0550	0.0250	0.455	4	0.500	0.500	0.000
High city	3.0450	1.4050	0.461	125	0.432	0.536	0.032
Step 1	0.0850	0.0300	0.353	6	0.667	0.333	0.000
Step 1+2	0.5800	0.3600	0.621	39	0.513	0.462	0.026
Step 1+2+3	0.3200	0.2025	**0.633**	44	**0.636**	**0.250**	0.114

The combined view clarifies the failure modes. Terminal PPO improves combat but nearly eliminates a behavior present in the imitation prior. Step 1 also keeps captures rare, so its apparently high $P(W \mid C)$ is not reliable because N_C is only six. High city has the opposite problem: it captures far more often, but those captures are more often associated with losses than wins, and Table 2 shows that this does not translate into stronger play. The meaningful comparison is therefore between Step 1+2 and Step 1+2+3. After reducing shaping pressure and adding the failed-city-attack penalty, Step 1+2+3 captures fewer cities, but succeeds on a larger fraction of attacks and its captures are more strongly associated with wins.

These capture-conditioned outcomes are diagnostic correlations, not randomized interventions. A policy may capture cities mostly when it is already ahead, or may be forced into city attacks when behind. We therefore do not claim that each observed capture causally increases win probability. The intended evidence is the combination of lower capture frequency, higher capture success, better capture-conditioned outcomes, and competitive head-to-head strength, which together suggest a shift from collecting city rewards toward more selective city use.

4.5 External Online and Heuristic Benchmarks

Internal method comparisons are necessary but not sufficient: a method can beat its own baselines while still being weak against external opponents. We therefore add two external anchors. First, we evaluate against Flobot, a public heuristic bot ported to the same local simulator interface. Second, we run online private-room games against Human.exe, a strong open-source community bot that prior work describes as the previous best-known Generals.io bot [10, 2]. These results are not as controlled as the local ablation, but they help calibrate whether the learned policy transfers beyond our retained method set.

Table 6: External benchmark summary. Local Flobot games use fixed 18×18 generated maps with swapped sides. Human.exe games are completed online private-room games on `bot.generals.io`; invalid websocket/startup attempts are excluded. Bold marks the best win rate within each opponent group.

Method	Opponent	Setting	Games	Record	Win rate	Interpretation
BC	Flobot	local fixed18	200	132–68	0.660	BC already beats the heuristic baseline
Terminal PPO	Flobot	local fixed18	200	185–13–2	**0.934**	Terminal PPO is far above Flobot
Step 1+2+3	Flobot	local fixed18	200	166–31–3	0.843	final staged method remains far above Flobot
Terminal PPO	Human.exe	online private room	18	3–15	0.167	Terminal PPO transfers poorly online
Step 1+2+3	Human.exe	online private room	40	16–24	**0.400**	final staged method is much closer to top-level play

The Flobot benchmark is mainly a sanity check: it shows that the learned policies are non-trivial and stronger than a public heuristic, but Flobot is too weak to separate the strongest methods. In particular, Terminal PPO scores higher than Step 1+2+3 against Flobot, so this benchmark should not be read as evidence for the city curriculum itself. Human.exe is a more informative external opponent. Terminal PPO wins only 3 of 18 completed online games against Human.exe, while Step 1+2+3 wins 16 of 40. The online sample is small, but the observed improvement suggests that the staged curriculum helps against a stronger external opponent, not only against retained local methods. Since prior work describes Human.exe as a previous best-known community bot and the

official Season 44 Week 1 ranking lists it at rank 31 in 1v1 play, a 40% score suggests that Step 1+2+3 is below but not far below top-level bot play [10, 3].

We also report a small online calibration against human opponents at different skill levels. This is useful because local self-play win rate does not directly translate into a public leaderboard rank. The goal of this calibration is to estimate a practical strength range, not to prove an exact rank. If an opponent has a reliable Elo-like rating R_{opp} and the agent obtains score p against that opponent, an approximate Elo anchor can be computed as

$$R_{\text{agent}} = R_{\text{opp}} + 400 \log_{10} \frac{p}{1-p}.$$

However, this estimate is meaningful only when the games use a consistent protocol and the sample size is large enough. Our current online records should therefore be treated as coarse anchors rather than precise leaderboard claims: Step 1+2+3 reliably beats casual players, wins most games against a mid-ranked player, and loses to Human.exe while still scoring 40%. The combined evidence is consistent with roughly top-100-level private-room strength, not an exact official leaderboard rank.

Table 7: Online strength calibration for Step 1+2+3. These private-room results are coarse practical anchors rather than a controlled leaderboard evaluation.

Method	Opponent group	Approximate level	Games	Record	Win rate / interpretation
Step 1+2+3	Casual players	beginner / unranked	13	12–1	0.923; stable wins over casual play
Step 1+2+3	Mid-ranked player	approximately top-500 level	15	11–4	0.733; likely above mid-ranked play
Step 1+2+3	Human.exe	rank 31 in Season 44 Week 1 1v1	40	16–24	0.400; below but competitive with top-level bot play

4.6 Qualitative Observations

Replay visualizations show the same pattern. Terminal-only agents often prioritize direct army concentration and general pressure, while city-shaped agents are more willing to invest armies into neutral cities. Step 1+2 frequently restores city investment, but can still spend armies on cities in tactically fragile positions. Step 1+2+3 is less aggressive about city capture and more often waits until the capture is supported by nearby territory or enough army, reducing low-quality crashes into neutral cities. Qualitatively, this is closer to the desired behavior: city capture is treated as a strategic investment, not as a local reward to collect whenever possible.

5 Discussion

The results suggest that terminal-only self-play can hide blind spots around delayed strategic objectives. A policy can win many local matches while ignoring a behavior that is important in broader human play. This is not necessarily a contradiction: if both self-play opponents share the same blind spot, the behavior may receive little learning pressure. Neutral city capture is one instance of this broader problem. It is measurable, important, and not immediately profitable: the policy must sacrifice current army for a future advantage, so the correct action depends on timing, local safety, and the expected length of the game.

The reward-shaping curriculum helps because each stage solves a different credit-assignment problem. Behavior cloning gives a stable starting distribution. PPO improves fighting. City-aware shaping makes the delayed investment visible to the optimizer. Lifecycle rewards discourage investments that cannot be held or survived. Finally, the low-shaping plus failed-city-attack-penalty stage filters out low-quality city crashes while keeping the agent under a strong win-oriented objective. This staged design is the main difference between simply adding an auxiliary reward and training a policy to use the delayed-return behavior at the right time.

The main limitation is that our shaping terms are heuristic. They are not potential-based and therefore do not preserve the optimal policy in the theoretical sense [4]. The approach is better viewed as curriculum design than as a clean objective transformation. The final stage also changes shaping scale and failed-city-attack feedback together, so our experiments support the combined stage rather than isolating which of these two changes is responsible. Another limitation is that the strongest causal evidence still comes from the local simulator. The Human.exe online games provide a useful external anchor, but 40 games are not enough to make a precise leaderboard ranking claim; broader online evaluation against humans and additional strong bots is still needed.

6 Conclusion

We trained Generals.io agents through behavior cloning and self-play PPO, and found that terminal-only self-play can produce strong agents that underlearn delayed strategic investments. We used neutral city capture as a measurable case study: it has an immediate army cost, delayed production return, and clear behavioral statistics. Staged reward shaping teaches when this costly investment may be worth its long-term return, then reduces shaping pressure and penalizes failed city attacks so that the policy re-optimizes for winning. The final Step 1+2+3 method improves head-to-head strength against several retained internal baselines while preserving more selective city capture behavior. This supports the broader lesson that sparse terminal rewards may over-credit fast combat outcomes and require carefully staged auxiliary signals to teach long-horizon risk-return behaviors in partially observable games.

A Implementation Summary

This appendix keeps only the implementation details needed to interpret the experiments. Full training configuration files live with the code; the paper focuses on the design choices that affect the method comparison.

Table 8: Compact implementation summary for the reported agents.

Component	Summary
Observation	8-frame history, 22 spatial planes per frame, concatenated over channels
Policy network	U-Net-style convolutional encoder with shared policy and value heads
Action space	Hierarchical pass/move policy with legal-move masking at inference
Auxiliary heads	Enemy-general prediction head exists in code but has zero loss weight for reported agents
BC	Three passes over filtered public replays; late-game pass labels are down-weighted
Self-play	PPO initialized from behavior cloning; all compared PPO branches use the same simulator setup
Evaluation	Fixed 18×18 local maps, swapped sides, 800-turn limit unless otherwise stated

Table 9: PPO hyperparameters for the reported self-play runs. The `updates` argument denotes target global update, not additional updates after resume.

Hyperparameter	Value
Optimizer	AdamW, weight decay 0
Learning rate	10^{-5}
Parallel environments	16 global environments
Rollout length	800 simulator turns per update
Episode cap during PPO	800 turns
PPO epochs / minibatch	4 epochs, global minibatch size 1024
Discount / GAE	$\gamma = 0.995$, $\lambda = 0.95$
Clip / target KL	clip range 0.1, target KL 0.02 for early stopping PPO epochs
Value / entropy / grad clip	value coefficient 0.5, entropy coefficient 0.001, gradient clip 1.0
Checkpointing	usually every 20 global updates for city-curriculum runs

Table 10: Training curriculum used for the main ablation. Names match Table 1.

Name	Purpose	Main difference from previous stage
BC	Imitation prior	Learn basic expansion and legal movement from replay data
Terminal PPO	Combat baseline	Optimize terminal win/loss reward without city-specific shaping
High city	Over-optimization control	Directly reward successful city captures with relatively large weights
Step 1	Preserve city behavior	Add smaller city rewards while retaining combat learned from BC/PPO
Step 1+2	Reward useful captures	Reward survival and holding the captured city over a fixed window
Step 1+2+3	Re-optimize for winning	Use 50% city shaping and penalize failed neutral-city attacks

B Reward and Opponent Summary

Table 11: Reward mechanisms used across the staged city curriculum.

Mechanism	Role in the curriculum
Terminal win/loss	The only reward in terminal-only PPO baselines
Game-stage schedules	Turn-dependent weights emphasize opening land, mid-game balance, and late-game strategic investment
Land milestone	Paid every 50 turns to encourage opening development
Material potential terms	Relative city ownership and army differences provide smoother learning signals
Enemy attack reward	Rewards damage to enemy-owned cells, not failed attacks on neutral cities
Neutral city capture	Paid only when a valid move successfully captures a neutral city
Lifecycle reward	Rewards surviving after capture and holding the city during a fixed post-capture window
Final penalty stage	The final stage multiplies stable city-shaping terms by 0.5 and penalizes failed neutral-city attacks

Table 12: Compact reward-curriculum summary. The table lists the reward differences that matter for the method comparison; all PPO stages retain terminal win/loss rewards.

Method	Initialization	Main city-related signal	Opponent sampling
Terminal PPO	BC	No city-specific shaping	Current-policy self-play / terminal PPO pool
High city	City-capture PPO parent	Direct successful-city-capture reward; late capture weights up to 0.36	Strong internal opponent pool
Step 1	BC	Conservative successful-city-capture reward; no lifecycle term	Mostly self-play during early city learning
Step 1+2	Step 1 branch	Lifecycle window $W = 100$: survival drip, hold drip, death penalty, and win-gated capture bonus	Mixed self-play and frozen internal policies
Step 1+2+3	Step 1+2	50% stable city shaping plus failed-city-attack penalty -0.02 after a 10-turn window	Mixed self-play plus strong frozen policies including the parent

Opponent sampling follows the same high-level pattern throughout PPO: early city stages are dominated by current-policy self-play, while later stages mix current self-play with frozen internal policies such as terminal PPO and earlier city-curriculum agents. The exact sampling weights are implementation configuration rather than a separate method contribution, but the staged comparisons above use the same fixed local simulator and PPO hyperparameter family.

Acknowledgements

This project was developed for the CS3308 machine learning course project. The implementation builds on public Generals.io replay data and was inspired by recent work on Generals.io reinforcement learning.

References

[1] Aaditya Bhatia, Austin Davis, Soumik Ghosh, and Gita Sukthankar. Generally genius: A Generals.io agent development and data collection framework. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 19, pages 400–406, 2023.

[2] EklipZ. Human.exe Generals.io Bot. `https://github.com/EklipZgit/generals-bot`, 2025. Open-source community Generals.io bot used as an online external benchmark.

[3] Generals.io. Generals.io season 44 rankings. `https://generals.io/rankings/44`, 2026. Official ranking archive; accessed June 2026.

[4] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 278–287, 1999.

[5] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention*, pages 234–241, 2015.

[6] Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 627–635, 2011.

[7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[8] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

[9] Matej Straka. Generals.io replays dataset. `https://huggingface.co/datasets/strakammm/generals_io_replays`, 2025. Accessed for supervised pre-training experiments.

[10] Matej Straka and Martin Schmid. Artificial generals intelligence: Mastering Generals.io with reinforcement learning, 2025.

[11] Huazhe Xu, Keiran Paster, Qibin Chen, Haoran Tang, Pieter Abbeel, Trevor Darrell, and Sergey Levine. Hierarchical deep reinforcement learning agent with counter self-play on competitive games. ICLR 2019 withdrawn submission, OpenReview, 2018.